

New York University
Computer Science Department
Courant Institute of Mathematical Sciences

Course Title: Cloud Computing

Course Number: csci-ga.3033-026

Instructor: Jean-Claude Franchitti

Session: 4

Course Project (Part I) – Application Cloudification/Migration

1. Application Installation:

In the first part of the course project, a reasonably complex business application is utilized as part of a case study (i.e., ArchNav) to identify the advantages and challenges of cloud migration. The ArchNav application consists of several components: a MySQL database, a Fortress application which utilizes a LDAP server for role based security, an ArchNav application that leverages an older web application framework, a Tomcat application server to house Fortress, and a Glassfish application server to house the ArchNav application. It is important to note that when viewing the portability to cloud for this application that the Fortress portion of the application should be considered a multi-tenant application. This means that it should be able to be utilized by other applications, not just this one, to manage user security configurations. The challenges to be addressed may have to do with cross-compatibility issues when installing the application itself and challenges faced when porting the application to cloud.

There are multiple components, which all require successful installation, for the ArchNav application to be deployed. The Windows 11 Operating System may be used in conjunction with Oracle Virtual Box (or equivalent) for installation on any platform. The application is also known to deploy successfully on Linux, CentOS, and MacOS (the installation on that last platform is known to be challenging). There are various steps required to install the ArchNav application and get it to deploy and run successfully. The steps provided below may be followed in a different order and are provided in an attempt to clarify the installation process:

The first step is to forward engineer the Archemy Schema model utilized by the ArchNav application and create the schema on a locally running instance of MySQL server (e.g., 127.0.0.1:3306). The second installation step, which is often the most challenging, is to configure the Apache Fortress application to run locally. The Fortress application has three main components, Fortress-core, Fortress-Web and Fortress-REST. The build manager Maven is used to build all three

of these Fortress projects. The Fortress-core project is the first to be installed and it generates a “fortress.properties” file used by the other Fortress components. The Fortress-Web and Fortress-REST components require a running LDAP server. SLAPD may be used on Linux and CentOS and it is best to use ApacheDS on Windows 10. The LDIF file downloaded during the Fortress-Core Git Pull is then executed by generating a connection to the locally running instance of ApacheDS (e.g., 127.0.0.1:10389). A similar process applies when SLAPD is used. The third installation step requires a successful build of the Fortress-Web and Fortress-REST components and their deployment on a locally running Tomcat Server. The web archive files generated by Maven when building each Fortress-Web and Fortress-REST application respectively need to be placed in the Tomcat webapps folder. After ensuring that the ApacheDS server is running on localhost:10389 (or equivalent for SLAPD), Tomcat needs to be started (e.g., on localhost:8080) to successfully deploy both Fortress-Web and Fortress-REST. The fourth step in deploying the ArchNav application requires the creation of a MySQL datasource to connect the application with the database via the Glassfish application server. A connection pool had to be configured to connect the application that runs on the application server with MySQL. The installation of an ADF essentials library in Glassfish is part of step 5 of the application setup. The next step requires using JDeveloper to build the ArchNav application as an “.ear” file and the final step is to deploy the corresponding .ear file to the Glassfish server.

From the steps presented above the installation of the ArchNav application is quite involved and challenges may be faced when attempting to install the application on different platforms. This issue is a key criterion when discussing migrating applications to cloud, as allowing for platform independence greatly simplify deployment across any cloud service platform (e.g., Amazon AWS, Microsoft Azure, IBM Cloud, GCP, Oracle Cloud, etc).

2. Application Cloudification/Migration:

Note: The suggestions provided in this section and the next section are purely indicative. You should suggest the migration approach that is most suitable at this time given the current state of the application and technologies available on the cloud.

Based on the challenges encountered when installing the ArchNav application locally there are many factors to consider as to whether the application should be migrated to the cloud and if so what would be the best way to do so. The main benefits that are obtained from a cloud migration are the elasticity of cloud that can be exploited. More specifically, if an application has various peaks in usage it may not be beneficial for a corporation to house their applications locally. Instead if the cloud is utilized corporations can simply pay as their application is used. However, there are also challenges of moving to cloud, including initial cost of migration and re-training of employees so the benefits of elasticity must outweigh these costs.

Assuming that the corporation utilizing the ArchNav application sees a benefit in moving to cloud, which you should assume is the case for this part of the project, the next consideration is how to migrate the application to cloud. As mentioned earlier, application platform independence would be crucial for this application due to its complexity, and it would also migration to a greater number of cloud service providers (e.g., Amazon AWS, Microsoft Azure, IBM Cloud, GCP, Oracle Cloud, etc.).

Several approaches may be considered to migrate the application to cloud. Two approaches

are being considered here for illustration purposes. One approach consists of deploying the Fortress and ArchNav applications directly to a cloud-based web application server. The MySQL portion of the application can then utilize the cloud-based storage to help with scalability. This approach follows the Platform-as-a-Service model and also has its drawbacks. These include the fact that the corporation migrating the application is limited to one cloud provider. If, for example, there is a new piece of functionality that can be used by the corporation at the Platform-as-a-Service level but with a different cloud provider, the application may have to be migrated all over again. Another challenge is that this application has many specific connection pools which may be hard to manage in the cloud. That said, using Platform-as-a-Service makes it easier to modernize and scale the application.

Another approach that may be considered to migrate the application is to use a container-based approach. Containers are lightweight and since some of the libraries needed by the application to run can be housed within containers, full platform independence may be achieved. This also includes the ability to easily move from one cloud platform to another. As an example, the application could be deployed using four containers: one for the MySQL server (assuming it is a correct choice), one for the Tomcat server housing Fortress-Web and Fortress-REST, one for the LDAP server, and one for the Glassfish application server housing the ArchNav application. The challenge with using (Docker) containers for this application is scalability due to the use of Java-based web servers such as Tomcat and Glassfish. One cannot simply deploy more instances of a container without the added complexity of correctly mapping ports to allow the application to properly run. One small tweak as it comes to this approach is to use the cloud storage option for MySQL with containers for the rest of the application which will help with horizontal scalability. Containers in this case are being used as part of the Infrastructure-as-a-Service as they are simply housing the application remotely. Furthermore, containers orchestration (via K8s) may be used to scale and harden the application deployment.

All options need to be considered before selecting a migration approach. In the two approaches suggested above, the platform independence of (Docker) containers and their ease of configurability (relative to Platform-as-a-Service) may be a good option for migrating the application to cloud. The approach would then be to build (Docker) containers for the Fortress-Web and Fortress-REST applications running on Tomcat, the ApacheDS LDAP Server, the Glassfish Server, and others as necessary. These containers can be built using pre-existing images as bases. For the Tomcat container the Tomcat-8 base image can be utilized and the Web Archive files for Fortress-Web and Fortress-REST then need to be deployed on top of the Tomcat-8 base image. Getting the Fortress-Web and Fortress-REST portions of the application to successfully connect to the ApacheDS server requires setting the IP appropriately within the fortress.properties file of the Fortress-Web and Fortress-REST applications to be able to connect to the Docker container running the ApacheDS LDAP server locally. However, this IP would need to be changed during an actual deployment. Alternatively, there could be a way to bind the container to a specific IP using Docker-Compose or the K8s equivalent. For the ApacheDS LDAP server container the base image to use would be ApacheDS. Sample Dockerfiles for these images can be found in the appendix below for illustration purpose.

To summarize, there are clearly many challenges faced as seen in this case study when it comes to installing and migrating a reasonably complex business application, such as ArchNav, to the cloud. The two aforementioned approaches suggested for porting this application to cloud are using Platform-as-a-Service and Containers as Infrastructure-as-a-Service. The Platform-as-a-

Service approach allows greater scalability but may limit the cloud migration to one cloud provider which may not be what the corporation that uses the application wants. Also, the ease of configuration for this application, in particular the connection pools in Glassfish may prove difficult for a Platform-as-a-Service model. Furthermore, the multi-tenancy aspect of Fortress could be lost if other cloud-based applications, which would like to utilize this role-based security, were hosted on different cloud providers. The other approach suggested to port this application to cloud is to use (Docker) containers. Containers fall into the category of Infrastructure-as-a-Service and provide some distinct advantages such as platform independence when migrating to cloud (this includes any cloud provider you migrate to). Furthermore, the separation of portions of the application into different containers as explained earlier may make the property of multi-tenancy for Fortress easier to enable. Configuring the Docker containers seems feasible and using them provide a viable migration approach for such an application. Software-as-a-Service does not seem very feasible for this application, unless the corporation that implements the application is able to find a Software-as-a-Service provider whose pre-configured application meets their needs. Business-Process-as-a-Service may not be applicable for this application given the fact that it is a complex business application. Potentially a portion of this application could be migrated to a SaaS or BPaaS cloud platform but that would still require configuring the remainder of the application.

The porting of the ArchNav to the cloud as part of this case study illustrates the many challenges and advantages that corporations must consider when migrating to cloud. Clearly, a thorough cost analysis is also advisable to see if the long-term benefit of cloud warrants the upfront cost. (Docker) containerization provides a very effective means of porting applications to the cloud. However, the ultimate best method for migration is up to the corporation's needs and in this case to the group of students working on the project.

3. Possible Cloudification/Migration Steps (for illustration purposes only):

Note: Please consider the current versions of the software packages supported for the version of the application you are migrating to cloud as you may be working with a more recent version of the application.

The cloudification of the Archemy ArchNav Application involves four major components. These components are an ApacheDS server to house the LDAP data, a Tomcat server that contains the Fortress-Web and Fortress-REST components, a Glassfish server to deploy the archemy.ear application and lastly a MySQL server. A combination of platforms may be used to move the application to the cloud. Amazon AWS and IBM Cloud are considered in the following. Each platform needs to be tested with various components of the application as it is typically the case that certain platforms are better suited for selected components. The following suggests a way to install the application in the cloud on a component-by-component basis (note that this is just an example provided for illustration purposes and is may not be the approach you want to follow):

ApacheDS Server component:

1. The ApacheDS service can easily be run in a Docker container by using the stock docker Image for ApacheDS. This image can be downloaded using the following command:

```
docker pull apacheds:latest
```

2. Subsequently run the image with the following command

`docker run <image_name_from_1>`

The image name from can be determined using: `docker ps -a`

3. Use a container registry on a cloud of your choice to push the image to the cloud.
4. Install Apache Directory Studio and execute the `apacheds-fortress.ldif` (See the ArchNav installation instructions for configuration details) on the instance of the ApacheDS Container in the container registry. Be sure to bind a public IP and note down this IP for it is necessary in installation of the subsequent components.

Tomcat Server with Fortress-Web and Fortress-Rest Installed component:

1. The Tomcat Server which houses the Fortress-Web and Fortress-REST components can be easily run in a Docker container by building a custom image which can be cloned from an available git repository (e.g., `git clone https://github.com/skyline220/tomcatdocker`).
2. Update the `fortress-web.war` and `fortress-rest.war` files by updating the `fortress.properties` files in each directory and re-building with that of the ApacheDS server. (See Archnav installation instructions by for configuration details).
3. Build the new image and push the image to a cloud container registry of your choice.
4. Run the container and ensure both the `fortress-web.war` and `fortress-rest.war` components install correctly

MySQL server component:

1. Select a cloud solution to run (the equivalent of) MySQL in the cloud.
2. Configure the server (you will have to update the schema name in both the SQL and stored procedures file) to the schema name of your cloud database instance. Use the ArchNav Installation instructions to install the server.

Archemy.ear deployment on Glassfish (3.1.2.2) component:

1. Create an Amazon AWS account and create an EC2 Instance of Ubuntu by following the instructions on Amazon.
2. Install java JDK 1.8 and set the `JAVA_HOME` environment variable in the EC2 instance by using the following instructions:

<https://www.digitalocean.com/community/tutorials/how-to-install-java-with-apt-get-on-ubuntu-16-04>

3. Add an inbound rule to your EC2 instance so the Glassfish Server can be accessed on ports 4848 and 8080.

4. Install Glassfish 3.1.2.2 on your EC2 instance by using the following

instructions or more recent documentation available:

<https://www.nabisoft.com/tutorials/glassfish/installing-glassfish-31-on-ubuntu>

5. Follow the ArchNav installation instructions to configure the Glassfish Server and deploy the archemy.ear application (Be sure to update the LDAP server address in the archemy- security portion).

4. Sources:

<https://docs.docker.com/>
https://en.wikipedia.org/wiki/Platform_as_a_service
<https://aws.amazon.com/eks/>

5. Appendix: (the Dockerfiles provided are provided for illustration purposes only)

1. *Sample Dockerfile for Glassfish*

```
## *- docker-image-name: "aot-java" *-  
# AoT  
#  
# Version 1  
FROM glassfish:latest  
MAINTAINER xyz  
"xyz@nyu.edu" COPY  
archemy.ear /  
#COPY start.sh /  
EXPOSE 8080  
EXPOSE 4848  
#ENTRYPOINT ["/start.sh"]
```

2. *Sample Dockerfile for Tomcat*

```
FROM tomcat:8.0  
# Maintainer  
MAINTAINER xyz@nyu.edu  
# Copy tomcat-users.xml  
ADD tomcat-users.xml /usr/local/tomcat/conf/  
# Copy catalina.sh with JAVA_OPTS  
ADD catalina.sh /usr/local/tomcat/bin/  
# Copy JAR  
ADD fortress-realm-proxy-2.0.0-RC1.jar /usr/local/tomcat/lib/  
# Copy images to tomcat path  
ADD fortress-rest.war /usr/local/tomcat/webapps/  
ADD fortress-web.war /usr/local/tomcat/webapps/  
EXPOSE 8080  
# Start the Service
```

CMD ["catalina.sh", "run"]

3. The standard ApacheDS image may be used for the ApacheDS Container